



Cairo University

Faculty of Economics and Political Science

Socio-Computing Department

***C201: Computer Applications in Social Sciences II***

# MODULE 2: PYTHON FUNDAMENTALS

## LECTURE 2\_3

### ***FUNCTIONS AND MODULES***

DR. AMAL SAKR

# OUTLINES

## Python Fundamentals

*Previous Lectures:* Variables, Operators, and Expressions, Condition (if...Else), and Loops (For and While Statements)

## **This Lecture: Functions and Modules**

- What is a Function?
- Python Built in Functions
- User-defined Functions
- Variable Scope
- Importance of Functions in Programming
- How to Write a Clean Code
- Module: Definitions, Built-in and User-defined Modules
- Packages

# What is a Function?

Functions in Programming are blocks of code that execute a specific task or related group of tasks.

There are two types of functions: *Built-in Functions* and *User-defined Functions*

## Built-in Functions

Functions that are already defined in Python libraries and can be called directly.

## User-defined Functions

Functions that are written by any of us to perform custom tasks specific to an application's requirements.

# Python Built in Functions

Python has a set of built-in functions. Below are some of these functions

- Python provides several important built-in functions that we can use without providing the function definition.
- The creators of Python wrote a set of functions to solve common problems and included them in Python for us to use.
- You should treat the names of built-in functions as reserved words (i.e., avoid using `max` or `min` for example as a variable name).

| Function                          | Description                                                                     |
|-----------------------------------|---------------------------------------------------------------------------------|
| <a href="#"><u>abs()</u></a>      | Returns the absolute value of a number                                          |
| <a href="#"><u>input()</u></a>    | Allowing user input                                                             |
| <a href="#"><u>int()</u></a>      | Returns an integer number                                                       |
| <a href="#"><u>float()</u></a>    | Returns a floating-point number                                                 |
| <a href="#"><u>len()</u></a>      | Returns the length of an object                                                 |
| <a href="#"><u>max()</u></a>      | Returns the largest item in an iterable                                         |
| <a href="#"><u>min()</u></a>      | Returns the smallest item in an iterable                                        |
| <a href="#"><u>open()</u></a>     | Opens a file and returns a file object                                          |
| <a href="#"><u>pow()</u></a>      | Returns the value of x to the power of y                                        |
| <a href="#"><u>print()</u></a>    | Prints to the standard output device                                            |
| <a href="#"><u>range()</u></a>    | Returns a sequence of numbers, starting from 0 and increments by 1 (by default) |
| <a href="#"><u>reversed()</u></a> | Returns a reversed iterator                                                     |
| <a href="#"><u>round()</u></a>    | Rounds a numbers                                                                |
| <a href="#"><u>sorted()</u></a>   | Returns a sorted list                                                           |
| <a href="#"><u>str()</u></a>      | Returns a string object                                                         |
| <a href="#"><u>sum()</u></a>      | Sums the items of an iterator                                                   |
| <a href="#"><u>type()</u></a>     | Returns the type of an object                                                   |

# Example

```
# ----- Examples for built-in functions
numbers = [10, 20, 30, 40, 70, 90, 85, 55]

print(len(numbers))      # Output: 8 (number of elements in the list)
print(sum(numbers))      # Output: 400 (sum of all elements)
print(max(numbers))      # Output: 90 (largest number in the list)
print(min(numbers))      # Output: 10 (smallest number in the list)
print(sorted(numbers))   # Output: [10, 20, 30, 40, 55, 70, 85, 90]
print(type(numbers))     # Output: <class 'list'>
```

```
# ----- Get the user's birth year
birth_year = int(input("Enter your birth year: "))
current_year = 2025 # Example year
age = current_year - birth_year
print(f"You are approximately {age} years old.")
```

# User-defined Function

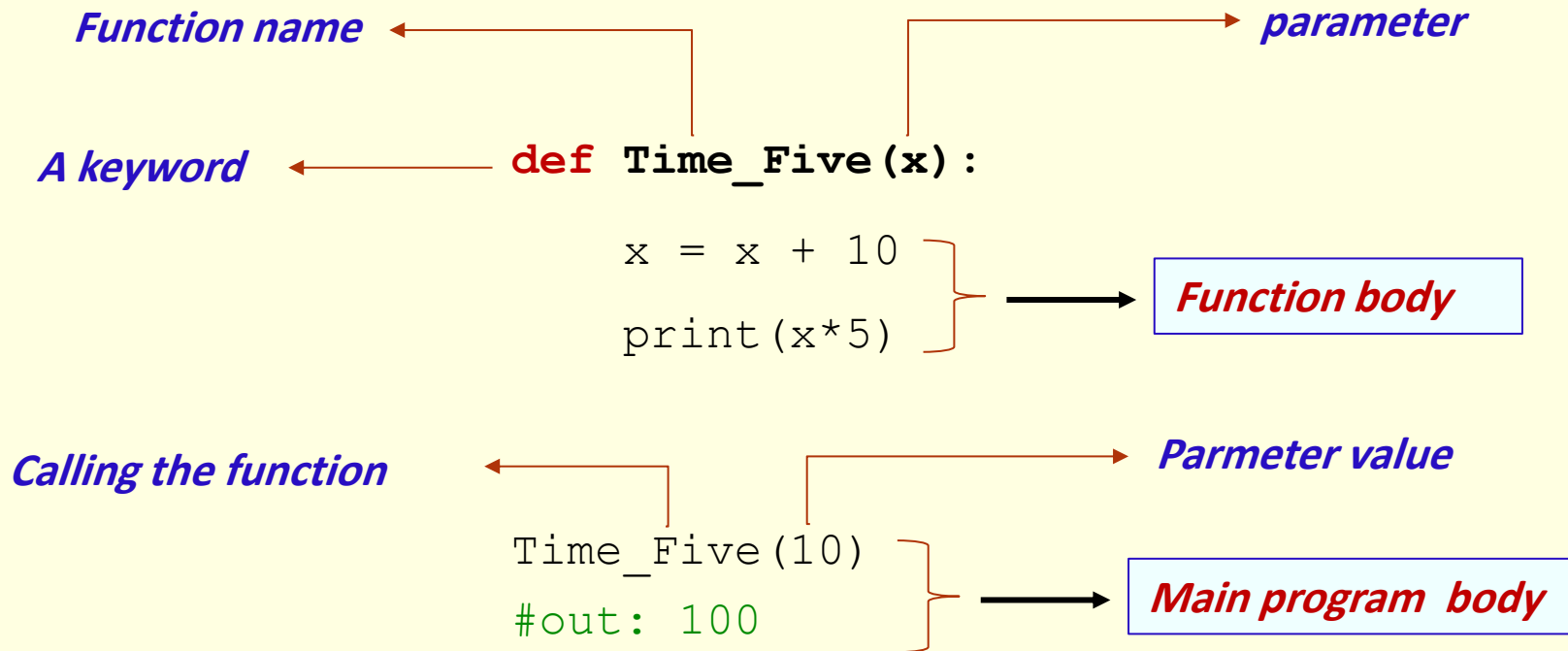
- When we create a function, we say we are *declaring* the function. When we ask the program to execute it, we say we are *calling* the function.
- Python provides the *def* keyword to define or declare a function.
- *Function\_name* is the name of the function. Function names follow the same rules as variable names.
- A function can be declared with or without *parameters (arguments)*. Use empty parentheses () after the function name if it has no arguments. The argument is a value or variable that we are passing into the function as input to that function.
- *return (optional)*: a function may or may not return a value when called.
- Defining a function **does not produce any output**; the function is executed only when it is called (or invoked).

## The Syntax of writing a function:

```
def function_name(list of parameters):  
    """  
        Comment statements explaining the function's behavior and  
        its parameters and return values. These are called Docstring  
    """  
    body of function's codes  
    return values  
  
# Calling a function  
function_name() # The function is executed when the function is called
```

# User-defined Function

Below is an example for a function called **Time\_Five** which receive a value, add 10 to it and print value\*5



**Note:** When reading a Python program, it's best to start with the **main part** where execution begins. Then check the functions when they are called to understand their role in the program. This helps you understand the program's overall flow and see which functions are called.

# Examples

```
#Print hello three times
def print_hello():
    print("Good Morning FEPS")
```

```
print_hello()
print_hello()
print_hello()
```

Main Program  
body

# Output

```
Good Morning FEPS
Good Morning FEPS
Good Morning FEPS
```

```
#A function which add two numbers
def add(a, b):
    s = a + b
    return s

r = add(5, 7)
print("Sum = ", r) #out: Sum = 12

print("Sum = ", add(20, 30))
#out: Sum = 50
```

**Function name:** `print_hello`

**Parameters:** No arguments

**Return value:** No return value

When the function is called, it prints the string "Good Morning FEPS".

In this example the function is **called three times**.

**Function name:** `add`

**Parameters:** Two parameters called `a` and `b`

**Return value:** `s`; the sum of `a` and `b`

When the function is called, it adds the two parameters' values

In this example the function is **called two times**: the first time where `a=5`, `b=7` and the second time where `a=20`, `b=30`

# Examples

```
# my_avg function receives two numbers and return their average
```

```
def my_avg(x,y):  
    return (x+y)/2
```

```
Result = my_avg(10,20)
```

```
    #call the function with two parameters 10 and 20, and save the  
    # return value in a variable called result
```

```
print(Result)                                #out: 15
```

```
print(my_avg(100,200))                      #out: 150
```

```
#call the function with two parameters 100 and 200 and print the  
#return value
```

```
# my_avg function receives a list of numbers and return their average
```

```
def my_avg(L):
```

```
    s=0
```

```
    for i in L:
```

```
        s=s+i
```

```
    return(s/len(L))
```

```
scores = [10,20,30,40,50]
```

```
av = my_avg(scores)    #call the my_avg function
```

```
print("Average using your function = ", av)                                #out: 30
```

```
print("Average using Built-in function = ", sum(scores)/len(scores))
```

```
#out: 30
```

# Examples

# A function that receives a number and prints if it is Even or Odd

```
def is_even (n):
```

```
    if n % 2 == 0:
```

```
        print(n, " is Even")
```

```
        return      # return stops the function, similar to break
```

```
    print(n, " is Odd")
```

```
    return
```

```
is_even(10)      # 10 is Even
```

```
is_even(7)       # 7 is Odd
```

```
"""
```

A function that converts an amount of money from one currency to another based on an exchange rate.

```
"""
```

```
def currency_converter(amount, exchange_rate):
```

```
    converted_amount = amount * exchange_rate
```

```
    return round (converted_amount,2)
```

```
print("Converted amount:",currency_converter(100, 1.2))
```

```
    # Out: Converted amount: 120.00
```

# Examples

Compute monthly loan payments based on *loan amount*, *interest rate*, and *repayment period*.

$$M = \frac{P \times r \times (1 + r)^n}{(1 + r)^n - 1}$$

$M$  = Monthly payment,     $P$  = Loan amount

$n$  = Number of months

$r$  = Monthly interest rate  $\left( \frac{\text{Annual Rate}}{12 * 100} \right)$

```
def loan_payment(principal, annual_rate, years):  
    months = years * 12  
    if annual_rate == 0: #If interest = 0, divide loan equally  
        return (principal/months)  
    monthly_rate = annual_rate/(100 * 12)  
    R = (1 + monthly_rate) ** months  
    payment = (principal * monthly_rate * R) / (R - 1)  
    return round(payment, 2)  
  
loan = loan_payment(10000, 5, 3)  
print(loan, "$ per month") # out: 299.71$ per month
```

# Examples

```
"""    The function takes no parameters. It gets 3 numbers from
the user and calculates their descriptive statistics """
```

```
def descriptive_stat():
```

```
    x = input("Enter your First Score: ");    x = float(x)
    y = input("Enter your Second Score: ");    y = float(y)
    z = input("Enter your Third Score: ");    z = float(z)
    print("Sum =", x + y + z);
    m = (x + y + z)/3;  print("Mean = ", round(m,2))
    v = ((x-m)**2 + (y-m)**2+ (z-m)**2)/3
    print("Variance = ", round(v,2))
```

```
descriptive_stat()#Read and describe the scores of the first student
```

```
descriptive_stat()#Read and describe the scores of the second student
```

Note: The function `descriptive_stat` ***does not get any parameters*** and ***does not return any value.***

# Examples: Different Techniques for Setting Parameters

```
# ----- Declaring a function that receives no parameters
```

```
def generate_full_name_1():  
    first_name = "Arwa"  
    last_name = "Mohammed"  
    full_name = first_name + ' ' + last_name  
    print('Full Name: ', full_name)
```

```
generate_full_name_1() # calling the function
```

```
# ----- A Function receives two parameters and return a Value
```

```
def generate_full_name_2(first_name, last_name):  
    full_name = first_name + ' ' + last_name  
    return full_name  
print('Full Name: ', generate_full_name_2("Arwa", "Mohammed"))
```

```
# A Function with Default Parameters and Different Calling Methods
```

```
def generate_full_name_3(first_name = 'Arwa', last_name = 'Mohammed'):  
    full_name = first_name + ' ' + last_name  
    return full_name
```

```
print(generate_full_name_3())           #Out: Arwa Mohammed  
print(generate_full_name_3('Mona', 'Saber')) #Out: Mona Saber  
print(generate_full_name_3(last_name="Saber")) #Out: Arwa Saber  
print(generate_full_name_3(first_name="Mona")) #Out: Mona Mohammed
```

# Examples: Different Techniques for Setting Parameters

**Static parameters** have a fixed number of arguments known when the function is defined. In contrast, **dynamic parameters** allow flexibility by adding `*` before the parameter name, enabling the function to handle any number of inputs at runtime.

```
# ----- Dynamic Parameters
```

```
def sum_all_nums(*nums):
```

```
    total = 0
```

```
    for i in nums:
```

```
        total += i        # same as total = total + i
```

```
    return total
```

```
print("Sum of all numbers = ",sum_all_nums(2, 3, 5)) #Out: 10
```

```
print("Sum of all numbers = ",sum_all_nums(2, 3, 5, 10, 20)) #Out:40
```

```
# One static parameter and several dynamic Parameters
```

```
def generate_groups (team,*args):
```

```
    print(team)
```

```
    j=0
```

```
    for i in args:
```

```
        j+=1
```

```
        print(f"Member {j}: {i} ")
```

```
    return
```

```
print(generate_groups('\nTeam-1', 'Ana', 'Mohammed', 'Mona', 'Saber'))
```

```
print(generate_groups('\nTeam-2', 'Ahmed', 'Nagi'))
```

```
print(generate_groups('\nTeam-3', 'Salma', 'Ameer', 'Mona'))
```

# Examples: Different Techniques for Setting Parameters

## # Output

```
Team-1  
Member 1: Ana  
Member 2: Mohammed  
Member 3: Mona  
Member 4: Saber
```

## # Output

```
Team-2  
Member 1: Ahmed  
Member 2: Nagi
```

## # Output

```
Team-3  
Member 1: Salma  
Member 2: Ameer  
Member 3: Mona
```

```
# ----- Function as a Parameter of Another Function
```

```
def square_number (n):
```

```
    return n * n
```

```
def get_power(f, x):
```

```
    return pow(f,x)
```

```
result = get_power(square_number(5), 3)
```

```
print("5 square to the power 3 = ",result) # Out: 15625
```

# Variable Scope

- ❑ A variable's scope defines where it can be accessed or modified.
- ❑ A variable can be classified into two types according to its scope:
  - **Local Variable**: Initialized inside a function, and it's accessible *only inside* its function. You get a *NameError* if you try to use a local variable outside its function.
  - **Global Variable**: If a variable is created in the main program (outside of all functions), then it is automatically a global variable. Which can be accessed by any function.
- ❑ To change the value of a global variable inside a function, you have to re-define it inside this function using the *global keyword*

```
# a function shows the diff between local and global variables
def print_var():
    y = 10      #y is a local variable
    z = x * 4
#inside the func. we can access x's value since it is a global var
    print(f"Inside func x = {x}, y = {y}, z = {z}") #5, 10, 20

x = 5  #x created in the main program → it is a global variable
print_var()
print("outside the function x = ", x)      #5
print("outside the function y = ", y)      # unacceptable "undefined y"
```

# Variable Scope

```
# a function shows how to update global variables
def print_var():
    y = 70
    #y is a local var different from global y which remains fixed = 10
    global x ;      x = -60
    # This will change x's value inside and outside the function
    print(f"Inside the function x = {x} and y = {y}")

x = 5; y = 10      # Two global variables
print(f"Before calling the function x = {x} and y = {y}")
print_var()
print(f"After calling the function x = {x} and y = {y}")
```

# ----- Output -----

Before calling the function **x = 5** and **y = 10**

Inside the function **x = -60** and **y = 70**

After calling the function **x = -60** and **y = 10**

# Recursion

Recursion refers to a function's ability to call itself. A function that calls itself is called a *recursive function*.

```
# Printing numbers from n to 0 using recursive function
def countdown(n):
    print(n)
    if n > 0:
        countdown(n - 1)

countdown(4)
```

## Trace

- $n = 4 \rightarrow$  prints 4  $\rightarrow$  calls `countdown(3)`
- $n = 3 \rightarrow$  prints 3  $\rightarrow$  calls `countdown(2)`
- $n = 2 \rightarrow$  prints 2  $\rightarrow$  calls `countdown(1)`
- $n = 1 \rightarrow$  prints 1  $\rightarrow$  calls `countdown(0)`
- $n = 0 \rightarrow$  prints 0  $\rightarrow$  since  $n$  is not greater than 0, recursion stops

## Output

4  
3  
2  
1  
0

# Recursion

```
# Calculating factorial of n using recursive function
def fact(n):
    if n <= 1:
        return 1
    else:
        return n * fact(n-1)

print(fact(4))
```

## Trace

- fact(4) → calc 4 \* fact(3)
- fact(3) → calc 3 \* fact(2)
- fact(2) → calc 2 \* fact(1)
- fact(1) → base case reached → returns 1

## Output

24

# Importance of Functions in Programming

Having studied functions, you are now prepared to comprehend their benefits.

## 1. Modularity of code:

Functions in Programming help *break down a program into smaller, manageable modules*. Each function can be developed, tested, and debugged independently, making the overall program more organized and easier to understand.

## 2. Abstraction:

Functions in Programming allow programmers to abstract the details of a particular operation. Instead of dealing with the entire implementation, a programmer can use a function with a clear interface, relying on its functionality *without the need to understand the internal complexities*. Functions hide the details of their operation, allowing the programmer to think at a higher level.

# Importance of Functions in Programming

## 3. Code Reusability:

Functions in Programming enable the reuse of code by *encapsulating a specific functionality*. Functions can be called multiple times, reducing code duplication which reduces redundancy and promotes efficient code maintenance.

## 4. Readability and Maintainability:

Well-designed functions enhance code readability by providing a clear structure and isolating specific tasks. This *makes it easier for programmers to understand and maintain the code*, especially in larger projects where complexity can be a challenge.

## 5. Testing and Debugging:

Functions make *testing and debugging much easier than large code blocks*. Since functions encapsulate specific functionalities, it is easier to isolate and test individual units of code. Debugging becomes more focused on a specific function, simplifying the identification and resolution of issues.

# Writing a Clean Code

## *Self-documenting code*

Self-documenting code is the code that is easily understandable and clearly states what it is meant to accomplish.

This includes

- Use meaningful names for variables, functions, classes, and modules.
- Write *small, focused* functions. Functions should have a *single responsibility* and be named accurately to reflect their purpose.
- Comments (statements start with **hashtag #**) provide an additional, useful explanation of the code. Commenting is a helpful technique that facilitates workflow documentation for you and other collaborators.
- Modularity and Self-documenting code improve teamwork, lower mistake rates, and promote trust.

# Writing a Clean Code

## *Self-documenting code*

```
# bad code
def process_transaction(x, y):
    if x >= y:
        execute_transaction(y)
```

```
# clean code
def process_transaction(user_balance, transaction_amount):
    # Check if the transaction can be processed
    if user_balance >= transaction_amount:
        execute_transaction(transaction_amount)
```

```
# better code
def process_transaction(user_balance, transaction_amount):
    if user_balance >= transaction_amount:
        execute_transaction(transaction_amount)
```

In last code, the comment is deleted because it is unnecessary. Use comments in a **wise way**.

# Module

## *Import Functions from User Defined Module*

- A module is a *single* file containing Python definitions and statements. These definitions and statements can include variables, functions, and classes. Module organizes and *reuses* code in Python by grouping related code into a single file.
- To create a module, write the codes in a python script, and save it as a **.py** file.
- Modules can be imported and used in other Python files using the *import* statement.

For example, create a file named **Module\_test.py** inside your project folder.

```
# Module_test.py file  
  
def generate_full_name(firstname, lastname):  
    print ("you are successfully important your module")  
    return firstname + ' ' + lastname
```

Create **main.py** file in your project directory and import the **Module\_test.py** file.

Import the file **Module\_test.py** inside **main.py**.

```
# main.py file  
  
import Module_test  
  
print(Module_test.generate_full_name("Mona", "Ahmed"))
```

# Module

## *Import Functions from User Defined Module*

```
# ----- main.py file... Calling Module_test.py file

from Module_test import generate_full_name, sum_two_nums, Interest_rate

deposit = 100000

profit = deposit * Interest_rate

print(generate_full_name("Ameer", "Kamal"))

print("Investment =", sum_two_nums(deposit, profit))
```

```
# ----- Module_test.py file

def generate_full_name(firstname, lastname):
    space = ' '
    fullname = firstname + space + lastname
    return fullname

def sum_two_nums (num1, num2):
    return num1 + num2

Interest_rate = .25 # Note that Interest_rate is just a variable
```

Note: The variable `Interest_rate` is initiated inside the module and accessed inside the main program. This is a valid action because we imported it in the main.

# Module

## *Import Functions from a Module and Renaming*

During importing we can **rename** the name of the module.

```
# ----- main.py file.. Calling Module_test.py file

from Module_test import generate_full_name as fullname,
sum_two_nums as total, Interest_rate as IR

deposit = 100000; profit = deposit * IR

print(fullname("Ameer", "Kamal"))

print("Investment = ", total(deposit, profit))
```

# Module

## *Import Built-in Modules*

- We can also import **built-in modules**. Some of the common built-in modules are *math*, *datetime*, *os*, *sys*, *random*, *statistics*, ...
- With *dir* or help built-in commands, it is possible to know the available functions in a certain module.
- Using Python *os* module it is possible to automatically perform many operating system tasks. The *os* module in Python provides functions for creating, changing current working directory, and removing a directory (folder), fetching its contents, and changing and identifying the current directory.

```
import os
import os
os.getcwd()           # Getting current working directory
os.mkdir('Khufu')      # Creating a directory named Khufu
os.chdir('d:/')        # Changing the current directory to d:/
os.mkdir('Cleopatra')  # Creating a directory named Cleopatra
os.rmdir('Cleopatra')  # Removing directory Cleopatra
os.rmdir('Khufu')
# Error: The system cannot find the file specified: Khufu
# because you have not changed the directory to its location
```

# Module

## *Import Built-in Modules*

The ***statistics*** module provides functions for mathematical statistics of numeric data. The popular statistical functions which are defined in this module: *mean, median, mode, stdev*,...etc. The **math** module contains many mathematical operations.

```
import statistics as st
```

```
ages = [20, 20, 4, 24, 25, 22, 26, 20, 23, 22, 26]
```

```
print(st.mean(ages))      # ~22.9
```

```
print(st.median(ages))    # 23
```

```
print(st.mode(ages))      # 20
```

```
print(st.stdev(ages))     # ~2.3
```

```
import math
```

```
print(math.pi)            # 3.141592653589793, pi constant
```

```
print(math.sqrt(2))        # 1.4142135623730951, square root of 2
```

```
print(math.pow(2, 3))      # 8.0, exponential function
```

```
print(math.log10(100))     # 2, logarithm with 10 as base
```

```
print(math.log(1))         # ln(1)
```

```
print(math.exp(0))         # e**0
```

```
print(math.factorial(4))   # 4!
```

```
print(math.floor(9.81))    # 9, rounding to the lowest
```

```
print(math.ceil(9.81))     # 10, rounding to the highest
```

# Module

## *Import Built-in Modules*

By now you are familiar with importing modules. Let us do one more import to get very familiar with it. Let us import `_random_` module which gives us a random numbers.

```
from random import random, randint
print(random())    # it returns a value between 0 and 1
print(randint(5, 20))
    # It returns a random integer number between [5, 20] inclusive
print(random.choice([1, 2, 3]))
    # choose a random element from a sequence
```

# Example: Lottery Function

```
# Lottery Function: Select Random Numbers without replacement
def lottery (min=0, max=100, draw_length=10):
    from random import randint

    if (type(min)!= int or type(max)!=int or type(draw_length)!=int):
        print("Wronge parameters types"); return
    if (max<min): print("Max should be greater than Min"); return
    if (max-min<draw_length): print("(Max-Min) should be greater than
        draw_length "); return
    d=[]
    while len(d)< draw_length:
        t=randint(min,max)
        if t not in d: d.append(t) #add an item to the list
    print(d)
    return d

# ----- calling Lottery Function by different ways
lottery()
lottery(5,20,6)
lottery(2,2,6)
lottery(-5,20.2,7)
lottery(max=30)
Rand = lottery(min=20,draw_length=5)
```

# Packages

- A Python package is a folder containing multiple Python modules that are grouped together to provide related functionality in an organized structure.
- To install a Python package, use the `pip install` command followed by the package name.

Example to install `numpy` package we write `pip install numpy`

- Every Python package needs an `__init__.py` file. It marks the folder as a package and can run setup code, such as importing submodules, defining variables, and setting default values.
- The command `pip list` displays a list of all Python packages currently installed in your environment, along with their version numbers.
- Beyond Anaconda, over *250 open-source packages* for data science and machine learning come pre-installed. For even more, explore the *Python Package Index* (<https://pypi.org/>).
- Whether someone works in a field like *Physics, Economics, game development, or natural language processing*, there is a strong chance a powerful library is already available to support it.

# Packages

Packages are imported using the *import* keyword, and modules or sub-packages are accessed with *dot notation*.

Below are some common packages and their conventional *aliases* used by data professionals:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

- *NumPy* is used for high-performance vector and matrix computations. *Pandas* is a library for manipulating and analyzing tabular data. *Seaborn* and *matplotlib* are both libraries used to create graphs, charts, and other data visualizations.
- Note that *matplotlib* is a **package**, and *pyplot* is a **module** inside that package, which is aliased as *plt*.

# Packages

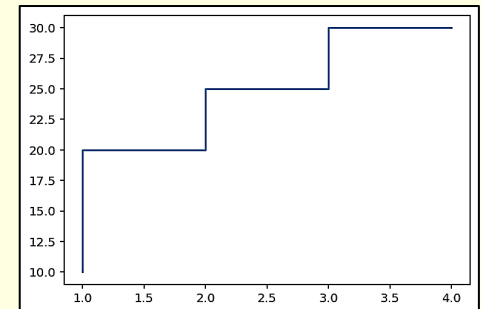
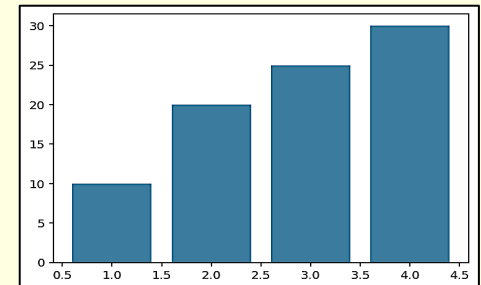
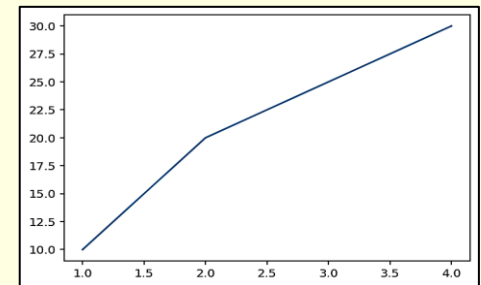
- After running these imports, whenever you want to use a function from one of these libraries, precede the function with the alias.

Example,

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]; y = [10, 20, 25, 30]
plt.plot(x, y)           #plot a line graph
plt.bar(x, y)
plt.step(x, y)
```

In the upcoming lectures, further information regarding these packages will be covered.

*Result Plots*



# Packages

From a **data science perspective**, there are three types of Python packages: *Operational, Data Visualization, and Machine Learning*.

Some packages can be used to accomplish the same tasks. However, they often have small differences that can make one more useful depending on the situation.

| Category                              | Purpose                                             | Common Packages                    | Key Features / Use Cases                                                                                                                                                                                                              |
|---------------------------------------|-----------------------------------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Operational Packages</b>        | Load, structure, and prepare data for analysis      | <b>pandas, numpy, sympy</b>        | <ul style="list-style-type: none"><li>— Read in data (e.g., <code>pandas.read_csv</code>)</li><li>— Preliminary data inspection</li><li>— Cleaning, merging, joining data frames</li><li>— Advanced mathematical operations</li></ul> |
| <b>2. Data Visualization Packages</b> | Create plots and graphs to explore and present data | <b>matplotlib, seaborn, plotly</b> | <ul style="list-style-type: none"><li>— Matplotlib: powerful, customizable, wide range of visualizations</li><li>— Seaborn: easy statistical plots</li><li>— Plotly: interactive dashboards &amp; publication-ready visuals</li></ul> |
| <b>3. Machine Learning Packages</b>   | Build and evaluate predictive models                | <b>scikit-learn</b>                | <ul style="list-style-type: none"><li>— Built on top of other Python libraries to provide tools to construct and analyze different types of models</li></ul>                                                                          |

# Code Deprecation

- Driven by an alive global community, Python is a living language—constantly evolving, innovating, and refining. Its ecosystem of libraries grows rapidly, shaped by real-world needs and user feedback.
- Over time, code evolves, and certain features may be eliminated or outdated—a process known as **deprecation**.
- In Jupyter notebooks, warnings signal deprecated features that may break in future versions. It's best to address them promptly to avoid future issues.
- While minor updates (e.g., *Python 3.11.8 to 3.11.9*) tend to have minimal impact, major version changes (such as *Python 2.x to 3.x*) often introduce significant deprecations, requiring closer attention and adaptation.
- To check which version of Python you're running, or a library you're using, execute the following code

```
import sys
print(sys.version)
#Out: 3.11.7
```

```
import numpy as np, pandas as pd
print(np.__version__) #Out: 1.26.4
print(pd.__version__) #Out: 2.1.4
```

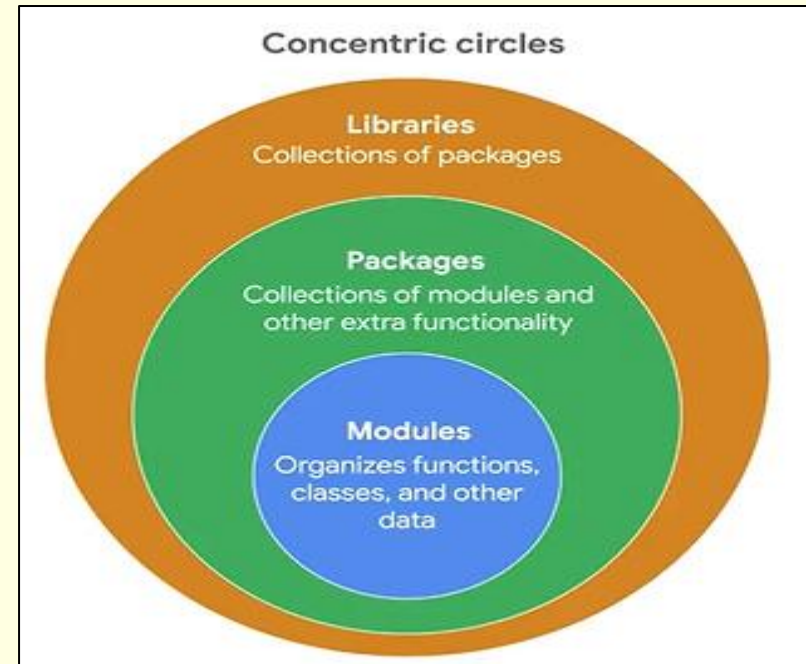
# Summary

**Functions** in Python are blocks of code that perform specific tasks and can be reused multiple times.

A **module** is a single file containing Python code, such as functions or classes, that can be imported into other programs.

A **package** is a collection of related modules organized in a directory hierarchy, allowing for structured and scalable code.

**Libraries** are broader collections of packages and modules that provide extensive functionality, such as data analysis, visualization, or machine learning, enabling developers to build programs efficiently without writing everything from scratch.



# Sheet 3: Ex 9

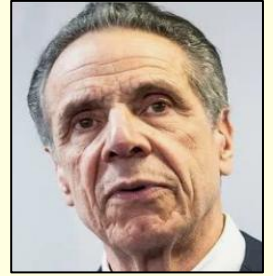
## Counts Occurrences of Two Specific Words in a List

Refer to questions 20 and 21 in sheet2, write one function to solve the two tasks. Name this function as `analyze_list`. The two questions are presented below

- Process a list of votes (e.g., "Mamdani" and "Cuomo") using a loop to determine the winner in the election (**Q20**).
- Take a list of gender responses ("Male" and "Female") and use a loop to calculate the percentage of each category(**Q21**).



*Zohran Mamdani*



*Andrew Cuomo*

### *Note:*

For now, we will assume the lists are predefined within the program.

After the next lecture on `Lists`, update the function to take a user-input list and ignore capitalization when comparing values.

# Answer

```
def analyze_list(data_list, word1, word2):  
    count1 = 0;      count2 = 0;      total = len(data_list)  
    for item in data_list: # Loop through the list  
        if item == word1: count1 += 1  
        elif item == word2: count2 += 1  
  
        # Display results  
    percent1 = (count1 / total) * 100  
    percent2 = (count2 / total) * 100  
    print(word1, ":", count1, "(", round(percent1, 2), "% )")  
    print(word2, ":", count2, "(", round(percent2, 2), "% )")  
  
    # Determine which one is higher  
    if count1 > count2: print(f"The Majority is {word1}.\n")  
    elif count2 > count1: print(f"The Majority is {word2}.\n")  
    else: print("The counts are equal\n")  
  
# Example 1: Gender list  
genders = ["Male", "Female", "Female", "Male", "Female", "Female"]  
analyze_list(genders, "Male", "Female")  
  
# Example 2: Voters list  
votes = ["Mamdani", "Cuomo", "Mamdani", "Mamdani", "Cuomo"]  
analyze_list(votes, "Mamdani", "Cuomo")
```

# *Sheet 3: Ex 8*

## *Calculating Exponential Growth (Decay) Over N Years*

Refer to questions 18 and 19 in sheet1, write one function to solve the two tasks. Name this function as `growth_model`. The two questions are presented below

- Use a while loop to model population growth for the next 10 years based on an initial population and a fixed growth rate per year (Q18).

*(initial\_pop += initial\_pop \* growth\_rate).*

- Simulate inflation by repeatedly increasing a product's price each year by a fixed percentage (Q19).

# Notes

The above two problems are like many real-life situations where quantities change over time by a constant rate rather than a fixed amount, meaning each period's change is proportional to the current value. The formula describing this change is:

$$\text{or } \textit{new value} = \textit{old value} + \textit{old value} \times \textit{rate}$$

$$P_{t+1} = P_t + P_t \times r = P_t(1 + r)$$

Repeated over several periods, it becomes

$$P_t = P_0(1 + r)^t$$

We call this **exponential growth** (or **decay** if  $r$  is negative) because the time variable ( $t$ ) appears in the exponent, making the quantity increase or decrease at an accelerating rate over time.

Examples include *population growth, inflation, and compound interest.*

# Answer

```
def growth_model(label, start_value=10000, growth_rate=.05, n=5):  
    year = 1;    value = start_value  
    print(f"Initial {label}: {value}")  
    while year <= n:  
        value += value * growth_rate  
        print("Year", year, ":", round(value, 2))  
        year += 1  
    print(f"After {n} years, the {label} = {round(value, 2)}\n")  
    return value
```

# Example 1: Population growth

```
growth_model("Population Growth", 1000, 0.02, 10 )
```

# Example 2: Inflation (price increases each year)

```
growth_model("Price Inflation", growth_rate = 0.08)
```

# Example 3: Compound Interest

```
growth_model("Deposit with Compound Interest")
```

# Example 4: Money Depreciation (value decreases each year)

```
growth_model("Money Depreciation", 1000, -0.15, n = 3)
```

# Sheet 3: Ex 10

## Calculating a Deposit Using Simple and Composite Interests

3. Write a Python program to *calculate* and *print* the growth of a deposit over time using simple and compound Interest rates. The final amount of money is calculated as follows:

|                            |                                 |
|----------------------------|---------------------------------|
| For Simple Interest Rate   | $A = P + (P \times r \times t)$ |
| For Compound Interest Rate | $A = P \times (1 + r)^t$        |

where,

|                                      |                                  |
|--------------------------------------|----------------------------------|
| $A$ = Final amount of money          | $r$ = Annual interest rate (0-1) |
| $p$ = Principal (initial investment) | $t$ = Time in years              |

Get the customer data:  $p$ ,  $r$ ,  $t$  from the user and create two functions

1. `Deposite_simple_interest(p, r, t)`

2. `growth_model(label="Deposit with Compound Interest", p, r, t)`

Place the last two functions in a module named `interest_growth` and import it into your main program. Use both functions to calculate and display the deposit growth.

**Note:** `growth_model` is the function from the previous example, and in this exercise, it is used to calculate the compound interest as previously explained.

# *Answer*

The `interest_growth` module file contains the below two functions:

```
def growth_model(label, start_value, growth_rate, n):
    year = 1;      value = start_value
    print(f"Initial {label}: {value}")
    while year <= n:
        value += value * growth_rate
        year += 1
    print(f"After {n} years, the {label} = {round(value, 2)}\n")
    return value

def Deposit_simple_interest(p, r, t):
    label = "Deposit with Simple Interest"
    value = p+(p*r*t)
    print(f"After {t} years, the {label} = {round(value, 2)}\n")
    return value
```

# *Answer*

The **main** file contains the below code:

```
import interest_growth

P = int(input("Principal investment: "))
r = float(input("Annual interest rate (0-1): "))
t = int(input("Time in years: "))

# Call the Deposit_simple_interest function from the module
interest_growth.Deposit_simple_interest(p, r, t)

title="Deposit with Compound Interest"

# Call the growth_model function from the module
interest_growth.growth_model(title, p, r, t)
```